

04

SQL Agent with Tool Use

Reason step-by-step over database tools

► think → act → observe → repeat

Doesn't write one query and hope. The agent is given three narrow tools — *list tables*, *inspect schema*, *run query* — and decides at each step which to call, reading the result before deciding its next move.

WORKFLOW

- 1

Think
The LLM is given the question and a list of available tools, with no schema up front. It decides what it needs to know first.
- 2

Act
It calls exactly one tool — *list_tables*, *get_schema(table)*, or *run_query(sql)* — never more than one per step.
- 3

Observe
The tool's output — a table list, a schema, a query result, or an error — is appended to the conversation as an observation.
- 4

Repeat or answer
The LLM re-reads everything so far and either takes another action or, once it has enough, stops and answers in plain English.

- VS REFERENCE
- Reference (and Experiment 1) generates a full query in one shot, against tables chosen by embedding retrieval. This agent doesn't know the schema at all until it asks — schema access is a tool call, not a precondition.
 - Failure handling isn't a separate repair function — a query error is just another observation the same loop reasons over, no special-cased retry path.
 - The trace is inherently multi-step and inspectable: every Thought/Action/Observation is a discrete, loggable turn, not two LLM calls hidden inside one function.

IMPLEMENTATION

sql_tool_agent.py

```
1  TOOLS = {
2      "list_tables": lambda conn: list_tables(conn),
3      "get_schema": lambda conn, table: get_schema(conn, table),
4      "run_query": lambda conn, sql: execute(conn, sql),
5  }
6
7  def react_loop(conn, question: str, max_steps=6):
8      history = [f"Question: {question}"]
9
10     for step in range(max_steps):
11         system_prompt = """You are a SQL agent. At each step choose ONE:
12         - list_tables()
13         - get_schema(table)
14         - run_query(sql)    # SELECT/WITH only
15         - answer(text)      # when you have enough to respond
16
17         Reply as: Thought: ...\nAction: tool_name(args)"""
18
19         response = call_llm(system_prompt, "\n".join(history))
20         thought, action = parse_action(response)
21         history.append(f"Thought: {thought}")
22
23         if action.tool == "answer":
24             return action.args["text"]
25
26         try:
27             observation = TOOLS[action.tool](conn, *action.args.values())
28         except SqlError as err:
29             observation = f"Error: {err}"
30
31         history.append(f"Action: {action.tool}({action.args})")
32         history.append(f"Observation: {observation}")
33
34     return "Could not resolve within step limit."
```

live demo → same warehouse as Experiment 1 – tool-calling loop replaces single-shot generation

SIDE-BY-SIDE ADDITION

EXPERIMENT	DATA FED TO THE LLM	DISTINGUISHING FEATURE
SQL Agent with Tool Use	Question only, upfront — schema discovered via tool calls	Multi-step Thought/Action/Observation loop; error is just another observation